

Real-Time Implementation of the PEMAFc Algorithm for Listening Technology Research

Bryan Monk

Department of Electrical and Computer Engineering

University of Illinois Chicago

Chicago, IL, USA

blmonk2@uic.edu

Abstract—Listening technology research often requires hardware setups that closely mimic practical conditions, including hearing aid systems with closely spaced microphones and speakers. This makes feedback cancellation a critical component, especially at high gain levels. This project presents the implementation and evaluation of prediction error method adaptive feedback cancellation (PEMAFC), a strong classical approach for acoustic feedback cancellation. Two variants, PEMAFc-f and PEMAFc-a, were implemented, corresponding to fixed and adaptive desired-signal models. Their performance was compared with the baseline of no feedback cancellation and with a basic feedback canceler that employed a normalized least mean squares (NLMS) update rule. Maximum stable gain was used as the primary evaluation metric.

I. INTRODUCTION

Acoustic feedback is one of the core problems that must be addressed in hearing aids. Because the microphones and speaker are separated by only a short distance, part of the amplified output can couple back into the input and be re-amplified. As the forward gain is increased, at a certain point the feedback becomes unstable which results in a sustained howling. The gain just before this occurs is called the maximum stable gain (MSG), which sets the limit on the usable gain for the system. This metric is especially important for hearing aid users with severe hearing loss who rely on high-gain devices.

There are many algorithms to choose from when approaching a feedback cancellation problem. Among the simplest and most common are adaptive filters that iteratively form an estimation of the feedback path that minimizes mean-squared error. These methods are appealing because of their relatively low complexity and straightforward implementation in real time. However, in practical systems, the desired input signal is usually correlated with the speaker signal, which can result in a biased estimate of the feedback path. This makes the problem more difficult than standard system identification and can result in poor performance if unaccounted for.

The prediction error method of adaptive feedback cancellation (PEMAFC) aims to reduce this bias by using a model of the desired signal in the adaptation. A prediction-error filter derived from that model is applied to the speaker and input signals to whiten them before the feedback path is estimated. This reduces the correlation that biases non-whitened approaches and leads to a more accurate estimate

of the feedback path if the incoming signal is close to our signal model.

The purpose of this project was to implement the effective but relatively simple PEMAFc algorithm on a real-time platform for hearing aid research, called the Tympan. This is an open source embedded audio development platform based around an ARM Cortex M7 microcontroller that can be freely programmed with user code. There is a matching earpiece for the Tympan that models closely a standard behind-the-ear (BTE) earpiece with a receiver-in-canal (RIC), and this earpiece can simply be plugged into the Tympan via the Earpiece Shield. To assess performance, the PEMAFc variants were compared against a baseline with no feedback cancellation, and a basic NLMS-based adaptive algorithm without whitening. Maximum stable gain (MSG) was the primary performance metric used. These efforts are in large part motivated by finding a suitable feedback cancellation algorithm for use in real-time research at UIC's Listening Technology Lab.

II. BACKGROUND

A. Feedback System Model

We first define a model for our system, whose configuration is shown in Fig. 1. The desired signal entering the microphone at discrete time index k is denoted by $x[k]$, while the total microphone signal is denoted by $y[k]$. The digital signal sent to the DAC and ultimately to the speaker is $u[k]$, and a portion of this signal is coupled back to the microphone through the feedback path.

The regular signal-processing path of the device is represented by the forward path $G(q, k)$, which is mainly a model of the time delay due to processing and the forward gain applied. The feedback path from the speaker to the microphone is represented by $F(q, k)$. This path includes any spectral alterations of both the speaker and microphone, the acoustic coupling between them, and any delays and frequency characteristics of the ADC and DAC.

The feedback path is modeled as a time-varying FIR filter with coefficient vector

$$\mathbf{f}[k] = [f_0[k] \ f_1[k] \ \cdots \ f_{L_F-1}[k]]^T,$$

so that its transfer function can be written as

$$F(q, k) = \mathbf{f}^T[k] \mathbf{q} = f_0[k] + f_1[k]q^{-1} + \cdots + f_{L_F-1}[k]q^{-(L_F-1)},$$

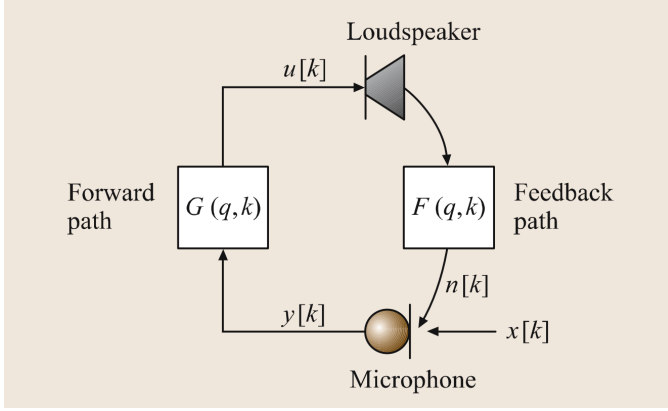


Fig. 1. Standard acoustic feedback model. Reproduced from [1].

where

$$\mathbf{q} = [1 \quad q^{-1} \quad \dots \quad q^{-(L_F-1)}]^T$$

and q^{-1} denotes the unit delay operator.

If the speaker signal is $u[k]$, then the feedback component arriving at the microphone is

$$n[k] = F(q, k)u[k].$$

Therefore, the microphone signal can be written as

$$y[k] = x[k] + n[k],$$

showing that the incoming signal consists of the desired signal plus the unwanted feedback component. The goal of each adaptive feedback cancellation algorithm is to continuously estimate the filter coefficients $\mathbf{f}[k]$ so that the contribution to $y[k]$ due to feedback can be subtracted out as much as possible.

B. Basic Adaptive Feedback Cancellation (AFC)

The simplest possible architecture for an AFC algorithm is shown in Fig. 2. This is known as the standard continuous adaptation feedback (CAF) technique.

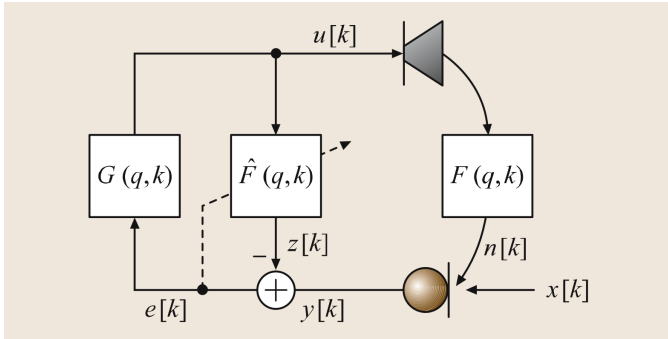


Fig. 2. Basic adaptive feedback cancellation. Reproduced from [1].

An adaptive filter $\hat{F}(q, k)$ is used to estimate the feedback path, and convolving with the speaker signal $u[k]$ results in the feedback estimate

$$z[k] = \hat{F}(q, k)u[k].$$

This is subtracted from the microphone signal to form the error signal

$$e[k] = y[k] - z[k].$$

Ideally, if $\hat{F}(q, k)$ matches $F(q, k)$, the feedback component becomes zero and the error signal approaches the desired external input, so that $e[k] \approx x[k]$.

The dashed arrow through $\hat{F}(q, k)$ indicates that the filter coefficients are updated using the available signals in order to improve the feedback estimate over time. This is done by minimizing the mean square or energy of the error signal. Since

$$e[k] = y[k] - \hat{F}(q, k)u[k],$$

the problem can be written as the minimization of the cost function

$$J = \mathbb{E}\{|e[k]|^2\}.$$

The feedback path coefficient vector $\mathbf{f}[k]$ has the corresponding estimate $\hat{\mathbf{f}}[k]$ so the feedback estimate can then be expressed as

$$z[k] = \hat{\mathbf{f}}^T[k]\mathbf{u}[k],$$

so that the instantaneous squared error becomes

$$|e[k]|^2 = |y[k] - \hat{\mathbf{f}}^T[k]\mathbf{u}[k]|^2.$$

In practice, we can't minimize J directly, since the true expectation is not available. Even if it was, it would not be a wise decision to use it because the feedback path can change over time. Instead, we use approximations of the expectation based on either instantaneous or shorter term values. Then, filter coefficients are updated recursively. For our standard CAF, the least mean squares (LMS) update rule for the filter coefficients can be formed by gradient descent with instantaneous approximations. Assuming real signals, this becomes:

$$e[k] = y[k] - \hat{\mathbf{f}}^T[k]\mathbf{u}[k]$$

$$J = \mathbb{E}\{e^2[k]\}$$

We can drop the expectation here to make the instantaneous approximation of J .

$$J_{\text{LMS}}[k] = e^2[k] = (y[k] - \hat{\mathbf{f}}^T[k]\mathbf{u}[k])^2$$

$$\nabla_{\hat{\mathbf{f}}} J_{\text{LMS}}[k] = -2e[k]\mathbf{u}[k]$$

Then the recursive weights update with step size μ becomes

$$\hat{\mathbf{f}}[k+1] = \hat{\mathbf{f}}[k] - \mu \nabla_{\hat{\mathbf{f}}} J_{\text{LMS}}[k] = \hat{\mathbf{f}}[k] + \mu e[k]\mathbf{u}[k].$$

Note that the factor of 2 is absorbed into μ . The LMS update can be improved upon because the size of the update depends on the input signal's energy. We can account for this and scale by this energy to arrive at the standard NLMS update rule:

$$\hat{\mathbf{f}}[k+1] = \hat{\mathbf{f}}[k] + \mu \frac{e[k] \mathbf{u}[k]}{\delta + \|\mathbf{u}[k]\|^2}$$

Here, δ is a regularization parameter that avoids division by zero and improves robustness/stability.

The standard CAF architecture unfortunately suffers from an important limitation that prevents the filter weights from converging to their true value. Because the speaker signal and desired signal are correlated, the estimate $\hat{\mathbf{f}}[k]$ can become biased. By solving for the optimal $\hat{\mathbf{f}}$ that minimizes our cost function we get:

$$\hat{\mathbf{f}} = \arg \min_{\hat{\mathbf{f}}} \mathbb{E} \left\{ \left(y[k] - \hat{\mathbf{f}}^T \mathbf{u}[k] \right)^2 \right\}$$

$$\hat{\mathbf{f}} = \mathbf{f} + \mathbf{R}_{uu}^{-1} \mathbf{r}_{ux}$$

where

$$\mathbf{R}_{uu} = \mathbb{E} \left\{ \mathbf{u}[k] \mathbf{u}^T[k] \right\}$$

is the autocorrelation matrix of signal $\mathbf{u}[k]$ and

$$\mathbf{r}_{ux} = \mathbb{E} \left\{ \mathbf{u}[k] x[k] \right\}$$

is the cross correlation between signals $\mathbf{u}[k]$ and $x[k]$. Here we can clearly see that a bias term is introduced if the correlation between these two signals is nonzero, and therefore the true value of the filter weights can never be estimated.

C. PEMAFc Approach

The biased estimate produced by the standard CAF setup is one of the main motivations behind the PEMAFc method. While other approaches can be taken to reduce bias, PEMAFc is shown to be one of the best performing and most reliable [1].

The PEMAFc method relies on a desired signal model, which is a model for how to filter a white noise excitation signal to produce the desired signal $x[k]$. Given a white noise signal $w[k]$ and the true desired signal model $H(q, k)$, the desired signal is written as

$$x[k] = H(q, k) w[k].$$

This means that the signal $y[k]$ can be written as

$$y[k] = F(q, k) u[k] + H(q, k) w[k].$$

The PEMAFc algorithm aims to minimize the prediction error signal rather than the direct error between $y[k]$ and $\hat{F}(q, k) u[k]$. The prediction error is defined as

$$e^f[k] = \hat{H}(q, k)^{-1} \left(y[k] - \hat{F}(q, k) u[k] \right),$$

where $\hat{H}(q, k)$ is our estimated desired signal model. We can see that if we plug in the expression for $y[k]$ and if our desired signal model estimate is accurate ($\hat{H}(q, k) = H(q, k)$), the error is minimized when the feedback path estimate is

correct ($\hat{F}(q, k) = F(q, k)$). In this case, only the white noise signal is left over so $e^f[k]$ is perfectly white. It is worth noting that

$$e^f[k] = \hat{H}(q, k)^{-1} e[k] = A(q, k) e[k],$$

where $A(q, k)$ is called the prediction error filter. We can now formulate a cost function with this new error. Again assuming real signals, we get

$$J = \mathbb{E} \left\{ \left(e^f[k] \right)^2 \right\}$$

$$\hat{\mathbf{f}} = \arg \min_{\hat{\mathbf{f}}} \mathbb{E} \left\{ \left(w[k] + (\mathbf{f} - \hat{\mathbf{f}})^T \mathbf{u}^f[k] \right)^2 \right\},$$

where $u^f[k] = \hat{H}(q, k)^{-1} u[k] = A(q, k) u[k]$ is the filtered version of the speaker signal. Minimizing this yields

$$\mathbf{r}_{u^f w} + \mathbf{R}_{u^f u^f} (\mathbf{f} - \hat{\mathbf{f}}) = \mathbf{0},$$

where

$$\mathbf{r}_{u^f w} = \mathbb{E} \left\{ \mathbf{u}^f[k] w[k] \right\}$$

and

$$\mathbf{R}_{u^f u^f} = \mathbb{E} \left\{ \mathbf{u}^f[k] (\mathbf{u}^f[k])^T \right\}.$$

Since $w[k]$ is white, $\mathbf{r}_{u^f w} = \mathbf{0}$. Assuming $\mathbf{R}_{u^f u^f}$ is invertible, this must mean $\hat{\mathbf{f}} = \mathbf{f}$ and thus the estimate is unbiased.

We can observe that the error can be written in the same form as the standard CAF, just with the filtered versions of the signals instead:

$$e^f[k] = y^f[k] - \hat{\mathbf{f}}^T[k] \mathbf{u}^f[k].$$

It follows that the NLMS update is then

$$\hat{\mathbf{f}}[k+1] = \hat{\mathbf{f}}[k] + \mu \frac{e^f[k] \mathbf{u}^f[k]}{\delta + \|\mathbf{u}^f[k]\|^2}.$$

The remaining question is: how do we come up with an accurate desired signal model so that we can achieve sufficient decorrelation? The speech spectrum can be modeled by an autoregressive (AR) model because it is typically highly correlated from sample to sample. A good starting place is the first order filter

$$\hat{H}(q, k) = \frac{1}{1 - \alpha q^{-1}}.$$

with $\alpha < 1$. Speech is more low-frequency energy than high-frequency energy, which is why a low-pass filter is a good model. When we take the inverse, which becomes the prediction error filter that we actually apply to the signals, its form is a convenient FIR filter. Note that if $\alpha = 0$, the NLMS update becomes equivalent to the standard CAF setup.

If we fix this first order filter, we arrive at the first PEMAFc variant, PEMAFc-f. The idea is to broadly model the speech

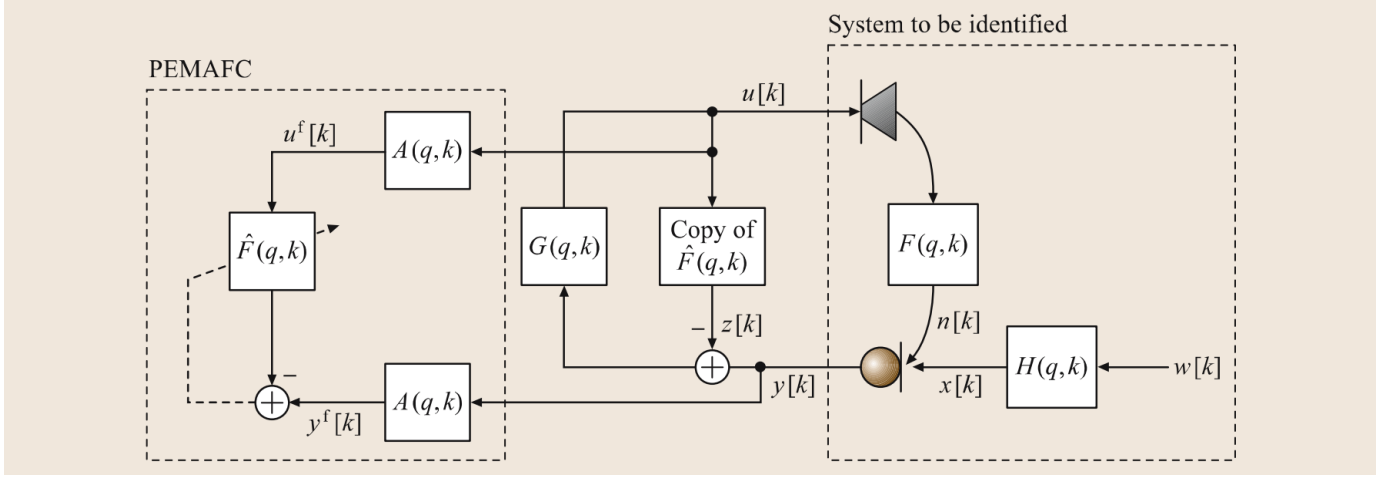


Fig. 3. PEMAFc model. Reproduced from [1].

spectrum in a fixed way for simplicity. In the second variant, PEMAFc-a, we take a step further and aim to adaptively estimate a more complex AR model that is more closely tailored to the specific incoming desired signal. Our AR model of order P for the desired signal can be written as

$$x[k] + \sum_{i=1}^P a_i x[k-i] = w[k].$$

Since the error signal $e[k]$ should closely resemble the desired signal, i.e. $e[k] \approx x[k]$, we can use $e[k]$ and its past values to solve the Yule-Walker equations, as shown below:

$$e[k] + \sum_{i=1}^P a_i e[k-i] = w[k]$$

$$r_e[\ell] = \mathbb{E}\{e[k]e[k-\ell]\}$$

$$\begin{bmatrix} r_e[0] & r_e[1] & \cdots & r_e[P-1] \\ r_e[1] & r_e[0] & \cdots & r_e[P-2] \\ \vdots & \vdots & \ddots & \vdots \\ r_e[P-1] & r_e[P-2] & \cdots & r_e[0] \end{bmatrix} \begin{bmatrix} -a_1 \\ -a_2 \\ \vdots \\ -a_P \end{bmatrix} = \begin{bmatrix} r_e[1] \\ r_e[2] \\ \vdots \\ r_e[P] \end{bmatrix}$$

Solving for the AR coefficients can be efficiently implemented with the Levinson-Durbin algorithm [4]. The PEMAFc-a algorithm should be able to more accurately estimate the true desired signal model than the fixed version. Given that the AR coefficients themselves are updated adaptively, it should also be able to respond to changes in this model, for example specific resonances that change as speech unfolds or differences in speech spectrum for different people.

III. REAL-TIME IMPLEMENTATION

As mentioned, the target system for this implementation was the Tympan, an open source embedded audio platform based on the Teensy 4.1 which contains an ARM Cortex-M7 microcontroller. Following the Tympan examples, an Arduino sketch was used to create the code that was ultimately flashed

to the system, while the actual audio processing code was created as a C++ library.

The library code begins with the function `update()` which is called each time an audio block comes in (16 samples in this case). If the feedback cancellation is enabled, it calls the `cha_afc()` function with pointers to the input and output blocks. This function is the main signal processing block.

Before delving into `cha_afc()`, it is first worth explaining the `receiveLoopBackAudio()` function which is what's called at the end of the audio chain before the signal is output to the speaker. This is necessary to retrieve and store our $u[k]$ signal, which also comes in a block of 16 samples, into a ring buffer.

Now that our $u[k]$ signal and its past values are available, we return to the `cha_afc()` function. For each sample in the audio block, we do the following:

- 1) Compute an offset for the ring buffer that stores $u[k]$ and its past samples.
- 2) Use that offset to compute the dot product between the filter estimate and $u[k]$, which results in the single-sample feedback estimate $z[k]$.
- 3) Form the error signal by subtracting the feedback estimate from $y[k]$.
- 4) Whiten the signal $y[k]$ into $y^f[k]$ by multiplying the prediction-error coefficients with past samples of $y[k]$.
- 5) Whiten $u[k]$ into $u^f[k]$ in the same way.
- 6) Form the whitened feedback estimate by taking the dot product of $u^f[k]$ and the estimated filter.
- 7) Form the prediction error $e^f[k]$ by subtracting the whitened feedback estimate from $y^f[k]$.
- 8) Perform the NLMS update. First, a dot product computes the squared norm $\|u^f[k]\|^2$. Then, the regularization parameter δ is added, and μ is divided by this sum and multiplied by $e^f[k]$. The resulting quantity is multiplied by $u^f[k]$ and added to the current filter to produce the next filter estimate.

After the NLMS update, we store the signal $e[k]$ in a

circular buffer. If the mode is set to PEMAFc-a and we've hit the specified AR block length, we then call the function `estimateARCoeffs()` which runs the algorithm to recompute the AR coefficients.

Estimating the AR coefficients has two steps: computing the autocorrelation function and then running the Levinson-Durbin algorithm.

First, the error signal $e[k]$ is accumulated in a buffer and the coefficient computation is triggered every AR block size. Note that this block size is much larger than the AR order so that autocorrelation measurement is as noise-free and accurate as possible, but still small enough that the AR coefficients update is not too infrequent. A fixed value of 320 was chosen, corresponding to about 13.3 ms at 24 kHz. The error signal is de-meanned over the AR block length and then the autocorrelation sequence $r_e[0], r_e[1], \dots, r_e[P]$ is computed by multiplying the error signal with delayed versions of itself. Finally, a small regularization is added to $r_e[0]$ to avoid division by 0.

Now we apply the Levinson-Durbin algorithm by calling `levinsonDurbin()` and passing through the autocorrelation and AR order, giving us the new estimated prediction error coefficients. The specifics of the Levinson-Durbin implementation will not be discussed, however it closely follows the procedure described by [4].

IV. TESTING AND RESULTS

The test setup involved placing the Tympan earpiece on the ear of a plastic model head. The two microphones on the Tympan were mixed equally in software. For the input signal, the author spoke toward the model, trying to keep volume and rate of speech consistent across measurements. The Tympan earpiece shield also includes a headphone output which was used to monitor the signal in detail to listen for subtle distortions. The step size was kept constant at $\mu = 0.001$, as this seemed to be a sweet spot between slow adaptation and audible distortion/instability. The NLMS regularization δ was fixed at 0.001 as well and the AR regularization set to $1e-5$. AR block length was fixed at 320 samples and the filter length set to 256 across all tests. It's also worth noting that the input gain was set to 20 dB, but this gain is a hardware gain for the ADC so it is not considered as part of the forward gain for the measurements. Gain was set low and increased in increments of 0.1 dB until the MSG was observed.

It is worth defining exactly what metric for MSG was used. The adaptive algorithms used in this project all seemed to be "stable" well beyond the point that the signal became distorted, meaning that the feedback loop would technically be suppressed to the point that the howling would not be consistently maintained. However, this is not too useful of a metric, because it ignores the severe distortion with howling characteristics that presents at lower gain. Therefore, MSG was defined similar to in [1], simply the point at which distortion was obvious, and would make the listening experience uncomfortable or annoying for human subjects.

Algorithm 1 Block-Based PEMAFc Update

Require: microphone block $y[k]$, speaker signal $u[k]$, feedback-path estimate $\hat{\mathbf{f}}[k]$ (initialized to $\mathbf{0}$)

Require: step size μ , regularization δ , feedback filter length L_F

Require: prediction-error coefficients $a_1, \dots, a_P$ (initialized to $\mathbf{0}$)

Loopback update

Store the newest speaker block in the speaker-history buffer so that past samples of $u[k]$ are available.

PEMAFc update

for each sample k in the audio block **do**

Form the speaker signal vector

$$\mathbf{u}[k] = [u[k] \quad u[k-1] \quad \dots \quad u[k-(L_F-1)]]^T.$$

Compute the feedback estimate

$$z[k] = \hat{\mathbf{f}}^T[k] \mathbf{u}[k].$$

Form the non-whitened error signal

$$e[k] = y[k] - z[k].$$

Apply the prediction-error filter to the microphone signal

$$y^f[k] = y[k] + \sum_{i=1}^P a_i y[k-i].$$

Apply the same prediction-error filter to the speaker regressor

$$\mathbf{u}^f[k] = \mathbf{u}[k] + \sum_{i=1}^P a_i \mathbf{u}[k-i].$$

Form the prediction error

$$e^f[k] = y^f[k] - \hat{\mathbf{f}}^T[k] \mathbf{u}^f[k].$$

Update the feedback-path estimate using NLMS

$$\hat{\mathbf{f}}[k+1] = \hat{\mathbf{f}}[k] + \mu \frac{e^f[k] \mathbf{u}^f[k]}{\delta + \|\mathbf{u}^f[k]\|^2}.$$

Store $e[k]$ in the AR estimation buffer.

end for

Adaptive AR update, PEMAFc-a only

if the AR estimation buffer contains one full AR update block **then**

Remove the mean from the stored block of $e[k]$.

Estimate $r_e[0], r_e[1], \dots, r_e[P]$ from the demeaned error signal.

Solve the Yule-Walker equations using the Levinson-Durbin algorithm.

Update the prediction-error coefficients $a_1, \dots, a_P$.

end if

With no AFC, the MSG was found to be approximately 32.3 dB. Added stable gain (ASG) is a metric that is often more useful for evaluating the performance of feedback cancellation algorithms since different setups can have a different baseline MSG. ASG is simply defined as the added gain from the no-AFC baseline, in this case $ASG = MSG - 32.3$. Both the MSG and ASG for each algorithm are shown in Fig. 4.

The standard CAF (or simply NLMS) was tested by setting the algorithm to PEMAFc-f mode with $\alpha = 0$. This means that the prediction error filter becomes $A(q) = 1$, a pass-through, so no whitening is done. Testing in this way ensured that everything besides the whitening was consistent across algorithms. As expected, the basic CAF performed better than the baseline, however it quickly became distorted, so the ASG was found to be 2.5 dB. This is unsurprising because the biased filter estimate guarantees that the estimated desired signal is incorrect which will manifest as audible distortions to the listener. Note that the gain could be increased much more before completely sustained howling occurred, but 2.5 dB was

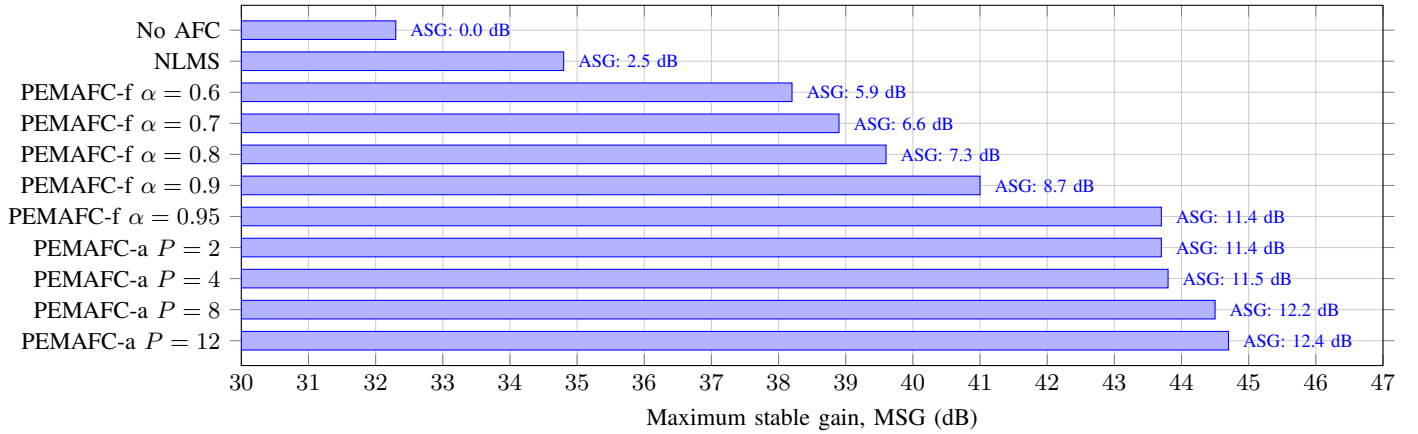


Fig. 4. Measured maximum stable gain for each adaptive feedback cancellation method. The labels show the added stable gain relative to the no-AFC condition which had a MSG of 32.3 dB.

the gain that met the criteria for MSG described previously.

PEMAFC-f showed considerable improvement over the standard CAF. Increasing α seemed to result in higher ASG, increasing from 5.9 dB at $\alpha = 0.6$ to 11.4 dB at $\alpha = 0.95$. A 10 dB increase is perceived as being twice as loud, so this is already very promising.

Testing showed that the PEMAFC-a algorithm could achieve even higher gains before distorting, specifically as AR order P was increased. An order of $P = 2$ resulted in similar performance to the best PEMAFC-f case (ASG=11.4 dB), which is expected since PEMAFC-f implements a 1st-order filter. PEMAFC-a reached a maximum ASG of 12.4 dB at $P = 12$. With this implementation of the PEMAFC-a algorithm, unfortunately the AR order could not be increased beyond $P = 12$ without exceeding the max CPU load.

It's also worth noting that on occasion, the PEMAFC-a variant would become completely unstable at high gain and result in sustained uncontrollable howling. This is likely to be attributed to the AR estimation becoming unstable, potentially during quiet periods where the adaptation is unproductive or due to insufficient regularization. Future work beyond the scope of this project will need to be done to address and debug this.

V. CONCLUSION

Overall, the implementation of the PEMAFC algorithm on a real-time device showed very good performance. Both the fixed and adaptive variants, PEMAFC-f and PEMAFC-a, resulted in a stark improvement over the regular NLMS algorithm. This proves that the employment of the prediction error filter before adaptation successfully decorrelates the speaker signal and desired signal, removing bias from the feedback path estimate to a noticeable degree.

However, both the algorithm and the specific implementation are not perfect. As discussed, the PEMAFC-a had some stability issues when compared with PEMAFC-f, likely due to the specifics of the AR adaptation algorithm. The PEMAFC-a algorithm will also need to be optimized to allow for

higher AR orders than $P = 12$. Additionally, PEMAFC is fundamentally limited by the ability to model the incoming signal as an AR model. Not every desired signal for a listener is highly correlated speech. Due to the tradeoff created by the AR block size, as the accuracy of the AR estimation increases, the adaptation speed decreases since it can only be updated so often. As these algorithms are linear, we also cannot model any nonlinearities that are no doubt present in hearing aid microphones and speakers to some degree. These effects only become more pronounced with increased gain, resulting in stronger harmonics that cannot be canceled.

One critical consideration not addressed is that when the feedback path changes at high gain, for example if a hand is placed in front of the system, there is loud howling until the filter estimate converges again. This edge case will likely need to be addressed independently, with something like a howling detection system and automated gain lowering until the filter reaches sufficient convergence.

Another important note is the inherent subjectivity when defining our MSG criteria. Since the different algorithms sound slightly different in terms of the distortions that they present, a true quantitative measure of MSG was not used, so others conducting the same tests may arrive at slightly different measurements. However, we must keep in mind that this is intended for human subjects, so a degree of human subjectivity is still valuable here.

There are many things that could be done in the future to characterize the algorithms better and improve upon them. Some useful measurements that were too complex for the scope of this project would be misalignment and convergence speed while tuning different parameters. This would involve measuring the feedback path by sending and receiving an impulse so that the true impulse response could be known at a given time. Different adaptation rules could also be tried such as recursive least squares (RLS). One improvement that would likely make a great difference is employing an adaptive adjustment of the step size μ . This would allow us to adapt less when there is silence and more when the signal is present.

Adapting the AR coefficients less when there is less signal might also be beneficial. Finally, the current design wastes some energy on predicting filter taps at the beginning that we know should be zero due to the inherent processing time and minimum delay of the feedback path. If we characterize exactly what this delay is, we can have the filter only adapt for the taps that matter.

Overall, this project is a great first step in an implementation suitable for real-time hearing aid research. After some adjustments and improvements, even high-gain experiments could be done with human subjects without fear of painful howling. UIC's Listening Lab will benefit from the ability to closely model real conditions used in practical hearing aids, newly enabling research that was previously impractical.

CODE LINKS

The code developed for this project is available in two repositories:

- C++ library code, included in a fork of the Tympan Library:
<https://github.com/username/repo>
- Arduino project used to run the experiments:
<https://github.com/username/repo>

REFERENCES

- [1] A. Spriet, S. Doclo, M. Moonen, and J. Wouters, "Feedback Control in Hearing Aids," in Springer Handbook of Speech Processing, J. Benesty, M. M. Sondhi, and Y. Huang, Eds. Berlin, Heidelberg: Springer, 2008.
- [2] A. Spriet, G. Rombouts, M. Moonen, and J. Wouters, "Adaptive feedback cancellation in hearing aids," *Journal of the Franklin Institute*, vol. 343, no. 6, pp. 545–573, 2006.
- [3] Tympan, "Tympan Library: Arduino/Teensy Library for Tympan Open Source Hearing Aid," GitHub. [Online]. Available: https://github.com/Tympan/Tympan_Library
- [4] C. Margina and B. Costinescu, "Implementing the Levinson-Durbin Algorithm on the StarCore™ SC140/SC1400 Cores," Freescale Semiconductor, Application Note AN2197, Rev. 1, Jan. 2005.